

Project 2

Open-Source AI Practice

(Deep Learning Programming Practice)

Kyuhong Shim

Dept. of Intelligent Software

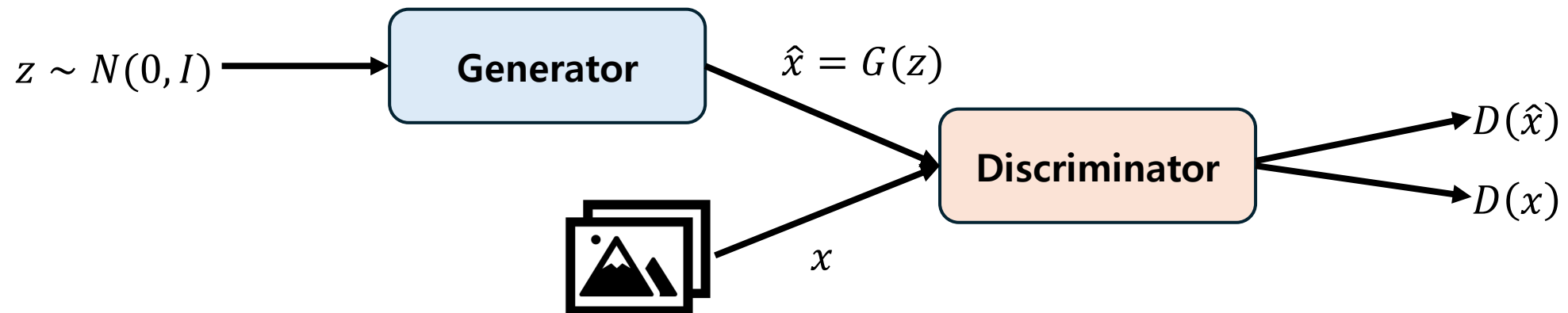
Sungkyunkwan University

2026. 5. 13.

Date	Notes	Date	Notes
3. 4.	Orientation	4. 29.	Project 1 QA
3. 11.	Tensor Assignment 02	5. 6.	Multi-task Learning Assignment 07
3. 18.	Gas accident (no class)	5. 13.	Project 2 (~6/9) Announcement
3. 25.	Module Assignment 03	5. 20.	Model Efficiency Assignment 08
4. 1.	Layer Assignment 04	5. 27.	Project 2 QA
4. 8.	Training Assignment 05	6. 3.	Election day (no class)
4. 15.	Project 1 (~5/5) Announcement	6. 10.	LLM Prompting Assignment 09
4. 22.	Optimization/Regularization Assignment 06	6. 17.	Best project presentation

• Image Generation using GAN

- Generative adversarial network (GAN) is a model that generates images from noise.
- GAN consists of Generator (G) and Discriminator (D).
- GAN uses adversarial loss:
 1. D should increase the D score for real images and decrease the D score for fake images.
 2. G should increase the D score for fake (generated) images.



- **Generate high-quality and realistic human faces**
 - Target: 1024x1024 image generation (FFHQ dataset)



- **FFHQ-1024**

- The dataset is high-quality, curated human face dataset [\[link\]](#).
- **Data split will be announced.**
 - 50k – training
 - 10k – valid
 - 10k – test
- **DO NOT USE VALID or TEST for training**
- Because the original images are too large (approx. 89GB for all), we will provide the **JPEG-compressed version** of the images.
 - This will cause JPG artifacts, but it is okay for this project.
 - Training and evaluation will both use the same JPG images.
- Do not use external datasets.

- **Generator**

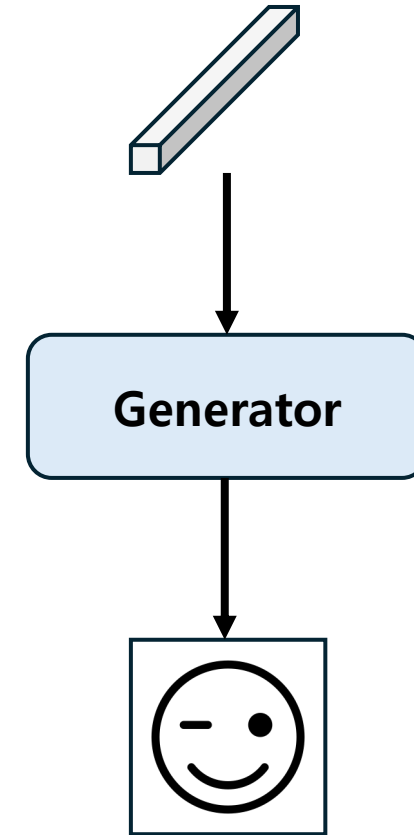
- The total parameter size should be **under 40M**.
- **Input: 512-dim random vector.**
- **Output: 3x1024x1024 RGB image.**

- **Discriminator**

- No strict requirements.
- D should be trained together with G.

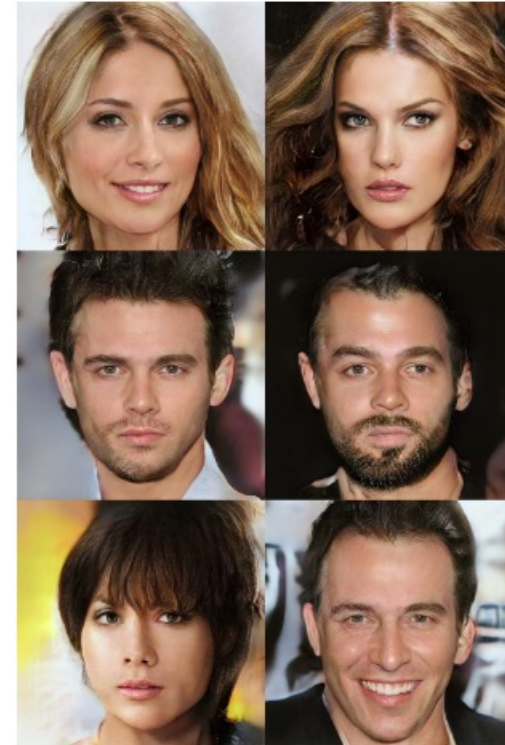
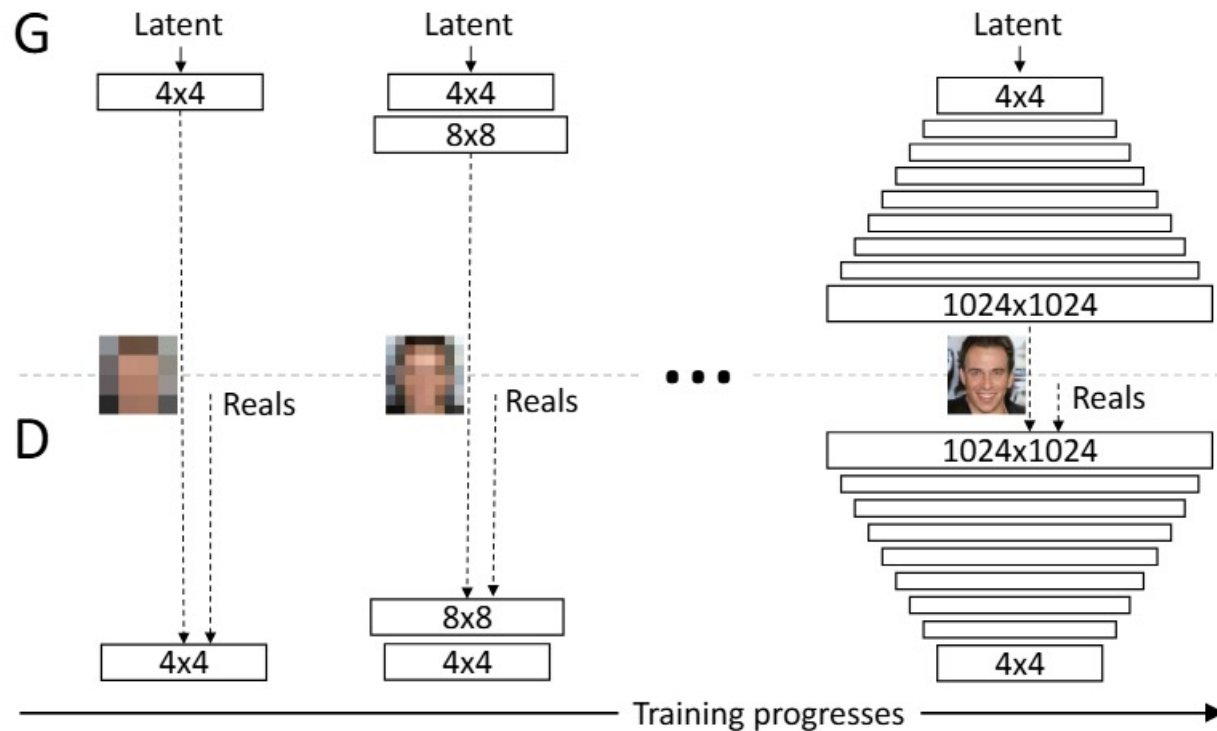
- Useful references

- BigGAN [[paper](#)]
- SA-GAN [[paper](#)]
- StyleGAN v2 [[paper](#)]
- StyleGAN v3 [[demo](#) / [paper](#)]

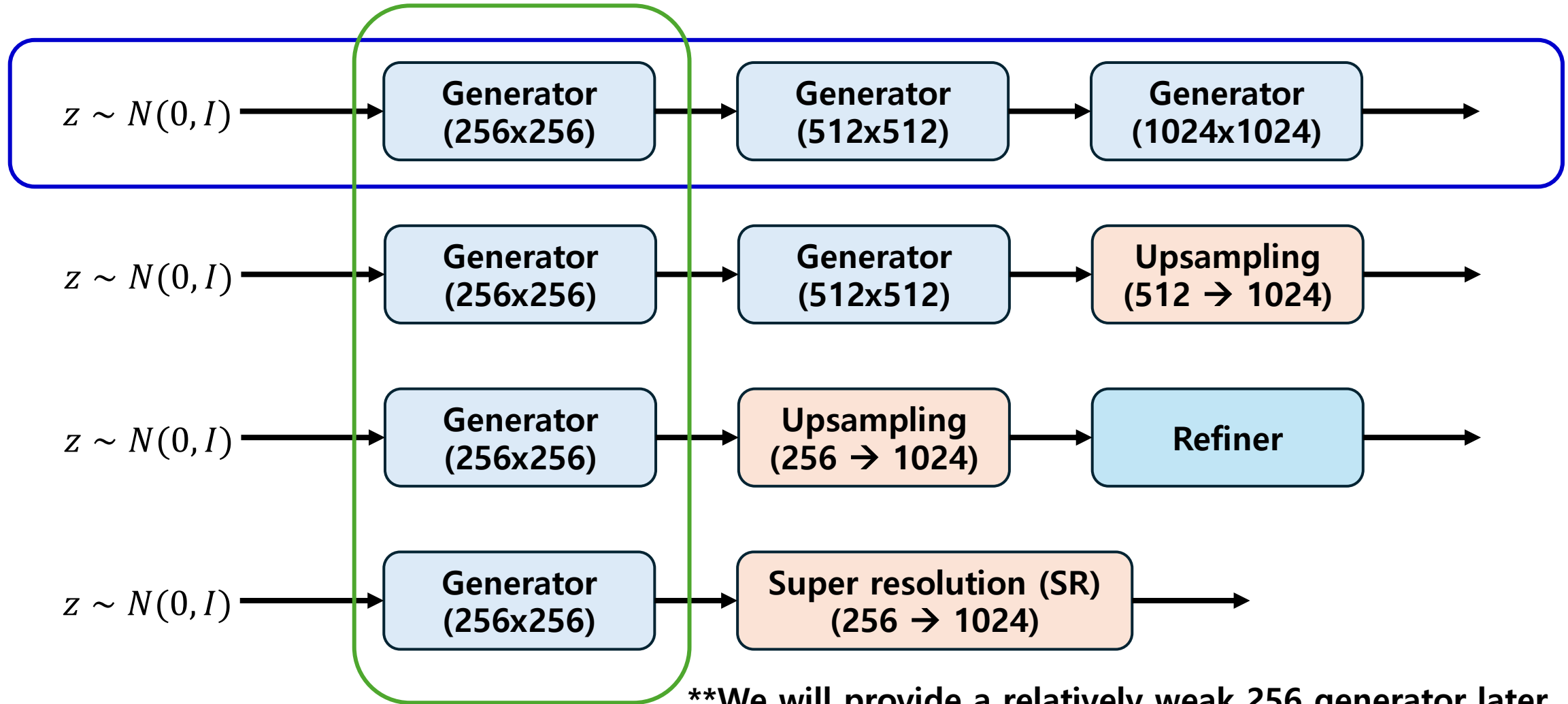


• Progressive Growing

- Training a high-resolution image from scratch is challenging.
- Progressive Growing (PG-GAN) [[paper](#)] starts from low-resolution images and progressively increases the resolution.
- In this project, we may adopt this strategy to $256 \rightarrow 512 \rightarrow 1024$.



- High-resolution strategies



- **Library**

- You can use:

- AI: **PyTorch** and **TorchVision**
 - Vision: OpenCV, Pillow, Scikit-Image, Matplotlib, etc.
 - Tracking: WandB
 - Evaluation: **pytorch-fid**

- Do not use:

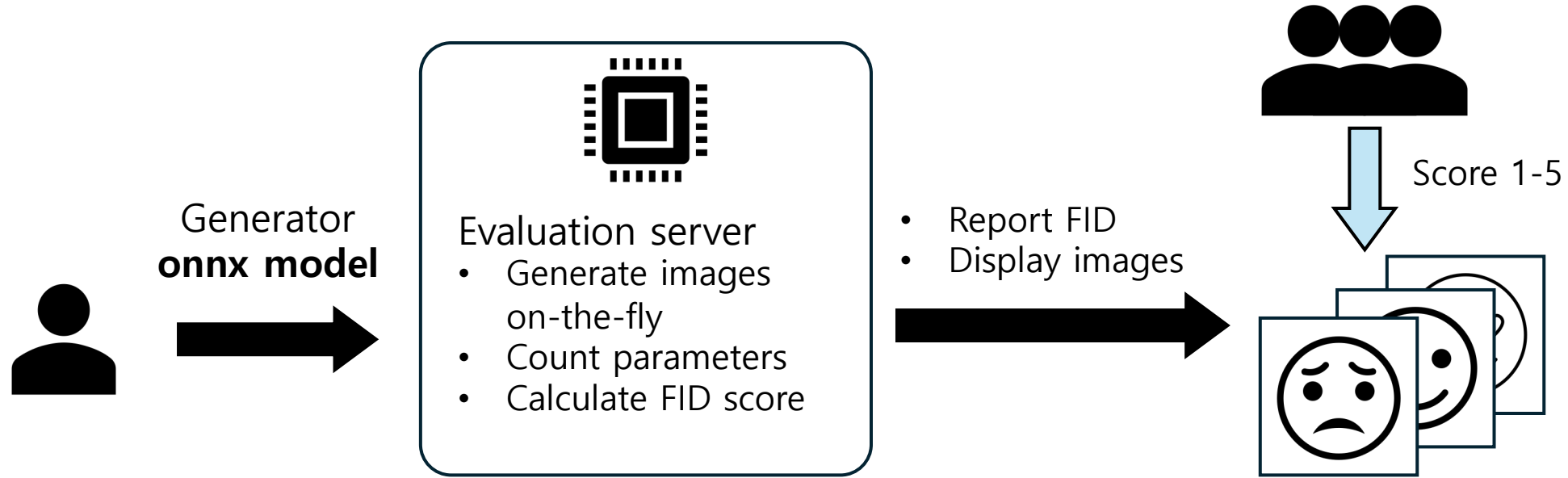
- [HuggingFace](#) libraries (Datasets, TIMM, Hub, Evaluate, etc.)
 - 3rd party libraries (PyTorch lightning, Accelerate, Albumentations, etc.) that support modeling, data processing, augmentation, training, or evaluation.
 - **DO NOT START FROM EXISTING REPOSITORIES / DO NOT USE PRETRAINED MODELS**
 - If you are not sure, ask professor ASAP.

- **AI Usage**

- You are encouraged to get help from ChatGPT, Claude Code, or Gemini.
 - In your report, briefly describe which AI tools you used and how they helped you.

- **The total score will consider both quantitative and qualitative measures.**
 - Quantitative
 - **FID score** (performance) (**10 points**)
 - Parameter count (**hard threshold: G under 40M**) (**5 points**)
 - Qualitative
 - Professor's quality assessment (**5 points**)
 - Students' quality assessment (**5 points**)
 - Code quality (**5 points**)
 - Report quality (**5 points**)

- **Code quality (not limited to):**
 - Clear module structure
 - Reproducible training and evaluation scripts
 - Proper checkpoint loading and saving
 - Readable implementation
 - No hard-coded test-set assumptions
 - Try to use many things learned in class.
- **Report quality (not limited to):**
 - Model architecture
 - Training recipe
 - Validation results
 - Ablation or trial history
 - Failure case analysis (wrong samples, weak classes, etc.)
 - WandB evidence (monitoring, config, overview, etc.)



- **Leaderboard will accept only one model per student.**
- 5/20 – 6/9: Professor and students may occasionally assign qualitative score. The model may change.
- 6/10 - 6/17: Qualitative scores will be reset, as the model should be fixed.

Due: 6/9 23:59

Submission

- Compress the entire codebase, **generator** checkpoint, and report into a single zip file.
- Do not need to submit the **discriminator**.
- Will announce how to submit your result.

• Important Notes

- Training data will be announced ASAP.
- Submission site and base 256x256 will be announced in the 5/20 class.
- Report should include WandB log and Overview page capture.
- We will check your WandB page during class, after project deadline.
- If you are not sure of anything (code, task, performance, data, etc), ask professor anytime.

(Zip file structure)

2025xxxxxx_project02.zip

- src/
- checkpoints/
 - model.pth
- 2025xxxxxx_project02_report.pdf
- pyproject.toml
- README.md

- README.md must include:
 - How to train and install dependencies.
 - How to generate the images from noise.
- Report
 - Report length must be no more than **6 pages**. Use an **11pt font size** for the main text. Submit the report in **PDF** format. There are no other formatting requirements.